

Engineering Write-Up: Autonomous Agentic AI System

1. System Overview & Design Decisions

The goal of this project was to design and build a functional, autonomous agent capable of goal decomposition, tool orchestration, live data retrieval, and graceful error recovery without relying on curated datasets.

- **Custom Plan-and-Execute Architecture:** Rather than depending on opaque, bloated black-box frameworks, the agent uses a modular Python class structure (AgenticSystem). This provides transparent control over the task lifecycle:

Planning → Execution → Self-Correction → Output Synthesis.

- **Live Tooling via Public APIs:** To respect the *No Provided Dataset* constraint, the agent utilizes the duckduckgo-search package (ddgs) to dynamically scrape live data from the web at inference time based on natural language user queries.
- **Structured Markdown Output:** The final synthesized data is automatically compiled and saved locally as agent_report.md, offering a clean, human-readable structure.

2. Robustness & Self-Correction Strategy

To satisfy the requirement for handling unexpected failures gracefully (weighted heavily in the evaluation rubric):

- **Validation Check:** After the web search tool executes, the system inspects the payload length and checks for error keywords.
- **Graceful Fallback:** If a tool returns weak, empty, or error-prone results, the agent intercepts the failure, logs a warning trace ([WARNING] Search tool returned weak results...), and initiates an automated **Self-Correction Routine** using a simplified fallback query (industry tech overview recent trends) to recover successfully without crashing.

3. Current Limitations

- **Deterministic Synthesis:** The local synthesis engine formats extracted text using structured templates rather than an advanced generative reasoning loop, which limits deep semantic summarization when underlying sources are noisy.
- **Single-Path Planning:** The initial plan is generated sequentially; if a mid-level step diverges significantly from user intent, the system lacks dynamic replanning loops to alter the remaining steps on the fly.

4. Future Improvements (With More Time)

- **LLM Integration with Fallback Parsing:** Integrate local open-source models (via Hugging Face or Ollama) or cost-effective API routing to enable fully generative planning and deep semantic text synthesis.
- **Dynamic Re-planning:** Expand the state machine so the agent can evaluate intermediate tool outputs and rewrite its remaining plan dynamically if a step yields unexpected context.
- **Expanded Toolset:** Add a Python safe-code execution sandbox and a calculator tool to handle multi-domain queries (e.g., automated data analysis or financial calculations).